

Database Schema

PostgreSQL · Developer reference · Source revision a76de2e

Assembly Adventures stores player progress and the question bank in PostgreSQL, hosted on UNC cloud apps. This document explains the tables, why they are shaped the way they are, and the rules the API tier has to follow.

- Initial DDL: `db/migrations/0001_init.sql`
- Gameplay additions: `db/migrations/0004_gameplay.sql`
- Local setup: [developer guide](#)
- Seed data: `db/seed/0002_levels_badges.sql`
- One-time question bootstrap: `db/seed/import_questions.py`

The database is the source of truth for questions. `questions.json` is a temporary file that predates it and is kept only as reference context; the importer exists to carry its 18 questions across once. After that bootstrap, questions are authored in the database, and both `questions.json` and the importer can be retired.

Why this exists

Before this schema, the entire state model was `player_data.gd`: a single in-memory `Array[String]` of earned badge names. Progress was lost on quit, the question bank in `questions.json` was never loaded by any code, and the one playable question was hardcoded in `levels/level_1/risc_v_level.gd`.

The schema exists to make three things possible:

1. Students log in with their Onyen and their progress follows them.
2. Questions are edited in the database, not shipped inside a game build.
3. Every answer is recorded, so we can see which questions students miss.

Current implementation

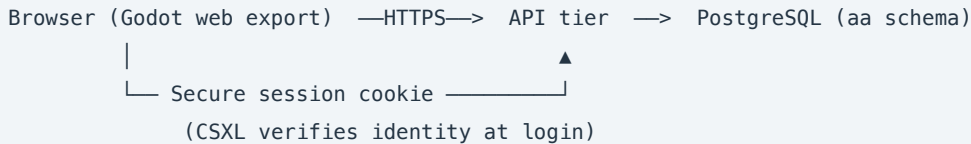
The deployed API and game now implement level 1 with all 13 questions, saved attempts, resume, and badges. Other levels are inactive. CSXL verifies Onyen and PID at login; no proxy identity headers are trusted. PID stays server-side. See [gameplay-api.md](#) for the implemented request contract.

Migration `0004_gameplay.sql` adds immutable `aa.run_questions` snapshots, per-attempt request IDs, one active run per player/level, PID validation, and a progress view that counts active badges consistently. Snapshot option IDs are randomized so source IDs such as `t1`, `t2` cannot reveal answer order.

Apply migrations through `python -m server.database`, which tracks names and checksums transactionally and never reapplies bootstrap seeds. The raw SQL remains authoritative. Use the tracked runner on both empty and existing databases.

Architecture

The game is a **Godot web export** served from **UNC CloudApps** behind the **FastAPI** server's **CSXL Onyen login and signed session cookie**. Because it runs in a browser tab, it can never hold database credentials, so an API tier sits between the game and Postgres and is the only thing that connects to the database.



The API tier is **FastAPI**. The raw SQL in `db/migrations/` remains the authoritative DDL; the API reads and writes those tables rather than generating its own schema.

Two rules are non-negotiable, because the browser is untrusted:

1. **Never send correct answers to the client.** `questions.json` ships answers inline, which was harmless when nothing was recorded. Now that answers are graded and badged, the endpoint that serves a question must strip `correct_option_ids`, `correct_order`, and each pair's `right` value.
2. **Grade and award server-side.** The client reports what the player did; the server decides whether it was right and whether a badge is earned.

Tables

`aa.players`

One row per authenticated UNC person, upserted from CSXL-verified attributes at login. There is no password column — the app delegates Onyen login to CSXL.

Column	Notes
<code>id</code>	<code>uuid</code> , the key used by every other table
<code>onyen</code>	<code>citext</code> , unique. Case-insensitive so lookups need no <code>LOWER()</code>
<code>pid</code>	9-digit UNC person ID. FERPA-sensitive
<code>role</code>	<code>student / instructor / admin</code> — the classroom hook

Handling `pid`. It is the only high-sensitivity column in the design. Never return it from the API, never write it to logs, and `REVOKE` it from any read-only reporting role. Everything downstream keys off `player_id` (the `uuid`), so analytics queries never carry an identifier.

PID is retained for the requested roster integration and never placed in the browser session cookie or returned by the gameplay API.

Classrooms are deliberately not modelled. `role` is the only hook. When an instructor view is needed, it becomes `courses` and `enrollments` tables plus a query — nothing already built has to change.

aa.levels

Column	Notes
slug	level_1 ... level_challenge, matching the question bank keys
scene_path	res://levels/..., replacing the hardcoded routing table
completion_rule	jsonb — what it takes to pass
is_active	false for levels that are still stubs

scene_path is stored as metadata. The current client routes Level 1 explicitly in `control_center.gd`; it does not dynamically load scenes from this column.

The only implemented completion rule is `{"type": "all_correct"}`. Other rules require server and client work before activation. Migration `0004_gameplay.sql` leaves only Level 1 active, even though historical seeds contain additional levels and questions.

aa.badges

Kept separate from `levels` rather than folded in as columns, for two reasons:

- Progress becomes `earned / count(active badges)`, which retires the hardcoded `/ 7` in `get_progress_ratio()`.
- An achievement that is not tied to a level (`level_id IS NULL`) needs no migration later.

slug deliberately matches the badge node names in `control_center.tscn` (`badge_1 ... challenge_badge`), since that string is already the de-facto progress key in the running game.

aa.questions

One table for all three question types, with the type-specific parts in a `payload jsonb` column. Shared columns cover `prompt`, `code_snippet`, `hint`, `explanation`, `points`, `difficulty`, and `ordinal`.

`code_snippet` exists because the source bank jams multi-line assembly into the prompt with `\n`. Splitting it lets the UI monospace the code and leave the prose alone.

Every payload element carries a stable id. This is the most important change from `questions.json`, which grades by comparing *option text*. Text comparison breaks on any copy-edit and makes "which wrong answer do students pick" unanswerable.

```
// multiple_choice
{"multi_select": false,
 "options": [{"id": "o1", "text": "4"}, {"id": "o2", "text": "5"}],
 "correct_option_ids": ["o2"]}

// drag_and_drop
{"orientation": "vertical",
 "tiles": [{"id": "t2", "text": "loop:"}, {"id": "t1", "text": "addi x5, x0, 3"}],
 "correct_order": ["t1", "t2"]}

// matching – the bank's positional fixed[i] ↔ answer[i] made explicit
{"columns": ["Role", "Register"],
 "pairs": [{"id": "p1", "left": "Destination register", "right": "x7"}],
 "distractors": []}
```

Drag-and-drop tiles are **stored in a scrambled order** so the seed file does not read as the solution top to bottom. The API must still shuffle per run — storage order is not a shuffle.

A `CHECK` constraint validates the payload shape per type. It is written with `→` rather than the `?&` operator on purpose: a bare `?` in DDL is treated as a bind placeholder by several drivers, which makes the constraint painful to re-emit from an ORM.

Questions are retired, never hard-edited. A substantive change inserts a new row and sets `is_active = false`, `retired_at = now()` on the old one, so old attempts still point at exactly what the student was shown. Typo fixes can be edited in place. The unique index on (`level_id`, `ordinal`) is partial — it only covers active rows, so a retired question does not block its replacement from reusing the slot.

aa.level_runs

One row per run through a level. A partial unique index permits only one `in_progress` run per player/level. Starting again resumes that run; starting after completion creates a new run. Wrong answers remain retryable and do not end the run. `lives_remaining` and `hints_used` are reserved schema fields, not implemented game mechanics in this release.

`questions_total` is captured at creation. Attempts record points once per question. Finishing checks all snapshots were solved, computes `questions_correct` and `score`, marks the run passed, and awards its badge.

aa.run_questions

Added in migration `0004_gameplay.sql`. Each (`run_id`, `question_id`) stores an immutable `public_question`, private `solution`, `explanation`, `points`, and `ordinal`. Public options and matching choices use fresh opaque IDs per run; source IDs and correct pair mappings are not exposed. Editing the question bank does not alter an active run's text or grading.

aa.question_attempts

The full attempt log — one row per answer submitted, including retries within a run (`attempt_no`). `response` mirrors the payload vocabulary, in ids:

```

{"selected_option_ids": ["opaque-option-id"]}           // multiple_choice
{"order": ["opaque-tile-id-2", "opaque-tile-id-1"]}     // drag_and_drop
{"pairs": {"opaque-row-id": "opaque-choice-id"}}       // matching

```

`player_id` is redundant with `run_id` → `level_runs.player_id`. It is kept so per-student analytics skips a join; the API sets it from the run, never from client input.

`elapsed_ms` is measured client-side, so treat it as **advisory** — a browser tab can be backgrounded, and browser timing is manipulable. It is a good signal for "which question makes people hesitate" and a bad one for grading. The API caps it at one hour before insert.

`request_id` is a client UUID with a unique (`run_id`, `request_id`) index. A retry with the same question and answer returns the previous result; different content under the same ID is rejected. Already-correct questions cannot earn more points.

aa.player_badges

Composite primary key on (`player_id`, `badge_id`), which makes re-awarding a no-op. Awards are made by the server; `player_data.gd` only caches the profile returned by the API.

Views

View	Purpose
<code>aa.player_progress</code>	<code>badges_earned</code> , <code>badges_total</code> , <code>progress_ratio</code> — replaces the / 7 denominator
<code>aa.question_stats</code>	Per question: attempts, distinct players, first-try correct rate, median time, hints used

`question_stats` is where keeping a full attempt log pays off, and it answers the instructor question without any instructor tables existing yet.

Implemented API

All routes live under `/api` and require the signed session cookie. Player registration happens in the verified CSXL login callback, not in `/api/me`. See [the API contract](#) for endpoint behavior, response formats, request deduplication, and errors.

Applying and evolving the schema

With the Python dependencies installed and database connection variables set:

```
python -m server.database
```

The runner applies migration and seed files in filename order under one transaction and advisory lock.

`public.aa_schema_migrations` records their checksums. It skips applied files and rejects edits to them. `0001` needs the `pgcrypto` and `citext` extensions; a managed database may require an administrator to enable these first. See [local setup](#).

Do not run `0003_questions.sql` manually or regenerate it for content updates: it is a one-time bootstrap with deletion logic. Add new reviewed SQL migrations for schema and question changes. Retire replaced questions instead of deleting history. Keep applied scripts in version control so fresh installations and existing deployments follow the same sequence.

There is no question-authoring UI or reporting endpoint yet. The database role column does not itself grant instructor/admin API functionality.

Known data issues in `questions.json`

The bootstrap importer fixes these automatically:

- The "drag-and-drop" key is misspelled in both `questions.json` and `questionBankDocs.md`. It maps to the `drag_and_drop` type.
- Two level 1 options have a stray trailing `]` ("`x6 is changed to 0]`", "`...at address x5 + 12]`"). Both are distractors, so stripping them does not affect answer matching.
- "`loop:` " in a drag-and-drop answer has a trailing space, which is stripped.

These need **manual attention** and are reported as warnings by the importer:

- **Four questions have code interleaved with prose**, e.g. "`Suppose x5 and x6 both contain 7.\nbne x5, x6, different\nWhat happens?`". The importer leaves these in `prompt` with `code_snippet` `NULL` rather than guessing a split that would reorder the question. Splitting them properly needs a second prose field (something like `prompt_after_code`) or an edit to the question text.

Resolved in the current implementation:

- The first Level 1 question now has `addi x5, x0, 12` in `code_snippet`, added by `0004_gameplay.sql`. The historical JSON still lacks that instruction.
- Level 1 no longer uses its old hardcoded question; it renders API snapshots.